



¿Qué es un Stack de Desarrollo?

Un stack de desarrollo, **también conocido como stack tecnológico**, se refiere al conjunto de tecnologías, herramientas y lenguajes de programación que se utilizan para crear una aplicación o sitio web. Esto incluye el sistema operativo del servidor, la base de datos, el servidor web, el lenguaje de programación del lado del servidor y el lenguaje de marcado del lado del cliente.

Importancia de Elegir el Stack Adecuado

Elegir el stack de desarrollo adecuado es crucial por varias razones:

1. **Eficiencia del Desarrollo:** Un stack bien elegido puede acelerar el proceso de desarrollo al proporcionar herramientas y bibliotecas adecuadas.
2. **Rendimiento:** Al seleccionar tecnologías optimizadas, puedes garantizar un mejor rendimiento de tu aplicación.
3. **Escalabilidad:** Un stack adecuado permitirá que tu aplicación se adapte a un aumento en la carga de usuarios y datos.
4. **Mantenimiento:** Un stack con una comunidad activa y soporte continuo facilitará el mantenimiento a largo plazo.
5. **Satisfacción del Cliente:** Al cumplir con los requisitos y expectativas del cliente, mejorarás la satisfacción del cliente.

EJEMPLOS DE ALGUNOS STACKS (PILAS) TECNOLOGICAS CONOCIDAS

Stack LAMP: Linux, Apache, MySQL, PHP/Python/Perl. Ampliamente utilizado para aplicaciones web.

Stack MEAN: MongoDB, Express.js, Angular, Node.js. Ideal para aplicaciones de una sola página (SPA).

Stack MERN: MongoDB, Express.js, React, Node.js. Adecuado para aplicaciones web modernas.

Stack JAMstack: JavaScript, APIs, Markup. Enfoque sin servidor para aplicaciones web estáticas.



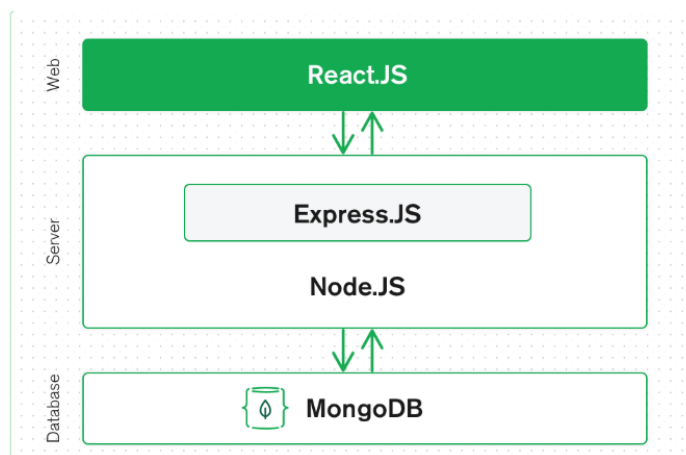
STACK O PILA: MERN

MERN es una pila tecnológica prediseñada basada en tecnologías JavaScript. MERN significa MongoDB, Express, React y Node , en honor a las cuatro tecnologías clave que la componen.

1. **MongoDB:** Es una base de datos NoSQL que almacena datos en formato JSON similar a documentos. Permite manejar grandes volúmenes de datos de forma flexible y escalable.
2. **Express.js:** Es un framework minimalista para Node.js que facilita la creación de servidores web y APIs. Maneja rutas y gestión de solicitudes/respuestas HTTP en el backend.
3. **React:** Es una biblioteca de JavaScript para construir interfaces de usuario interactivas y dinámicas en el frontend. Permite crear componentes reutilizables y manejar el estado de la aplicación eficientemente.
4. **Node.js:** Es un entorno de ejecución para JavaScript que permite correr código en el servidor. Así, se unifica el desarrollo tanto en frontend como en backend usando un solo lenguaje.

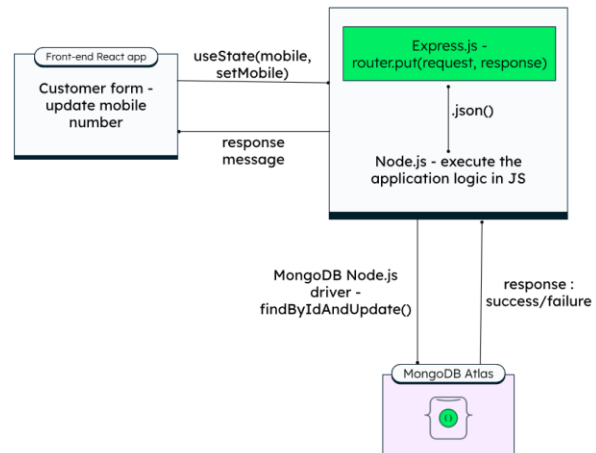
¿Cómo funciona la pila MERN?

La arquitectura MERN permite construir fácilmente una arquitectura de tres niveles (front end, back end, base de datos) utilizando completamente JavaScript y JSON.





EJEMPLO DE UNA SOLICITUD/RESPUESTA EMPLEANDO EL STACK MERN





BASES DE DATOS NO RELACIONALES

Las bases de datos no relacionales, también conocidas como **NoSQL**, se caracterizan **por no usar** un esquema tabular de filas y columnas como las bases de datos relacionales. Existen principalmente cuatro tipos de bases de datos no relacionales:

- **Bases de datos de clave-valor:** almacenan datos en pares clave (identificador) y valor, ideales para acceso rápido a datos simples.
- **Bases de datos de documentos:** guardan datos en documentos que pueden contener distintos tipos y formatos de datos, como JSON, y son muy versátiles.
- **Bases de datos orientadas a columnas:** almacenan datos organizados en columnas en lugar de filas, optimizadas para grandes volúmenes de datos y consultas analíticas.
- **Bases de datos orientadas a grafos:** especializadas en gestionar y consultar relaciones complejas entre datos, usando nodos y relaciones.

Cada tipo está diseñado para diferentes necesidades y tipos de datos, proporcionando flexibilidad y escalabilidad para aplicaciones modernas que manejan datos semiestructurados o no estructurados

CUADRO COMPARATIVO BASES DE DATOS RELACIONALES VS NO RELACIONALES

Aspecto	Base de Datos Relacional	Base de Datos No Relacional Basada en Documentos
Estructura	Organiza los datos en tablas con filas y columnas fijas.	Almacena los datos en documentos flexibles (p. ej. JSON, BSON) sin esquema fijo.
Modelo de datos	Datos estructurados con relaciones entre tablas.	Datos semiestructurados o no estructurados, cada documento puede tener una estructura distinta.
Lenguaje de consulta	Utiliza SQL para consultas complejas y uniones.	Usa APIs propietarias o lenguajes específicos, consultas menos complejas.
Integridad y consistencia	Sigue el modelo ACID, garantiza integridad estricta y transacciones atómicas.	Sigue el modelo BASE, ofrece coherencia eventual con alta disponibilidad.
Escalabilidad	Escalabilidad vertical, menos flexible para grandes volúmenes distribuidos.	Escalabilidad horizontal, adecuada para grandes volúmenes y datos distribuidos.
Flexibilidad	Menos flexible, requiere esquema rígido.	Alta flexibilidad en el esquema, fácil adaptación a cambios en datos.



Casos de uso comunes	Sistemas financieros, ERP, donde integridad y relaciones complejas son cruciales.	Aplicaciones web, redes sociales, IoT, gestión de contenidos con datos diversos.
Ejemplos	MySQL, PostgreSQL, Oracle.	MongoDB , CouchDB, Firebase Firestore.

MONGO DB

MongoDB es un sistema de gestión de bases de datos NoSQL orientado a documentos y de código abierto. En lugar de almacenar los datos en tablas y filas como en las bases de datos relacionales, **MongoDB guarda los datos en documentos con formato BSON (una versión binaria de JSON)** que permiten un esquema flexible y dinámico. Esto facilita la integración y administración de datos diversos y con estructuras variables. MongoDB es altamente escalable y ofrece un modelo de consulta avanzado, siendo muy utilizado para manejar grandes volúmenes de datos en aplicaciones modernas. También permite manejar colecciones de documentos que pueden tener diferentes estructuras, brindando gran flexibilidad y rendimiento para aplicaciones que requieren alta disponibilidad y escalabilidad horizontal.

INSTALACION

Descargaremos la versión local **COMMUNITY SERVER** y su cliente de administración grafico denominado **COMPASS** así:

<https://www.mongodb.com/products/self-managed/community-edition>

<https://www.mongodb.com/es/products/tools/compass>

DOCUMENTACION

<https://www.mongodb.com/docs/manual/>



TALLER PRACTICO 1.

- Instalación del gestor de base de datos
- creación de colección, creación de documento, llenado
- Consultas básicas (usar Chuleta de consultas rápidas)

RETO

Crear una base de datos denominada veterinaria

Crear una colección (collection) denominado mascota

Diccionario de datos mascotas reguladas

nombre	
tipo	Perro, gato, marranito pig, iguana
raza	Persa, doberman
edad	
color	
vacunas	Booleano para saber si las tiene

- Inserta un documento
- Inserta varios documentos
- Inserta documentos hasta alcanzar 20
- Filtra los documentos para las mascotas cuya edad sea mayor a 5
- Ordena los documentos por la raza de la mascota de forma alfabética descendente (a..z)

Copia cada uno de los comandos que resuelven el reto.